



نام درس: شبکه های کامپیوتری
نویسنده: محمد امانلو - ۸۴۰۰۰۰۱۰۸۱
مهراد لیویان - ۸۱۰۱۰۱۵۰۱



پروژه اول

مراحل نصب:

ابتدا فایل ۸ گیگی را دانلود می کردم و در پوشه مورد نظر قرار دادم:

```
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/1$ ls  
p4-vm.qcow2  
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/1$
```

سپس دستور زیر را وارد می کنیم تا آپدیت شود:

```
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/1$ sudo apt-get install qemu-utils  
[sudo] password for mehrad:  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
  libaio1t64 libboost-iostreams1.83.0 libboost-thread1.83.0 libdaxctl1 libiscsi7 libndctl6 libnfs14 libnuma1 libpnm1  
  libpnmobj1 librados2 librbd1 librdmacm1t64 liburing2 qemu-block-extra  
The following NEW packages will be installed:  
  libaio1t64 libboost-iostreams1.83.0 libboost-thread1.83.0 libdaxctl1 libiscsi7 libndctl6 libnfs14 libnuma1 libpnm1  
  libpnmobj1 librados2 librbd1 librdmacm1t64 liburing2 qemu-block-extra qemu-utils  
0 upgraded, 16 newly installed, 0 to remove and 75 not upgraded.  
Need to get 11.0 MB of archives.  
After this operation, 49.7 MB of additional disk space will be used.  
Do you want to continue? [Y/n] Y  
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 libnuma1 amd64 2.0.18-1build1 [23.3 kB]  
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libaio1t64 amd64 0.3.113-6build1.1 [7210 B]  
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libboost-iostreams1.83.0 amd64 1.83.0-2.1ubuntu3.1 [259  
kB]  
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libboost-thread1.83.0 amd64 1.83.0-2.1ubuntu3.1 [276 kB]  
Get:5 http://archive.ubuntu.com/ubuntu noble/main amd64 libdaxctl1 amd64 77-2ubuntu2 [21.4 kB]  
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 librdmacm1t64 amd64 50.0-2ubuntu0.2 [70.7 kB]  
Get:7 http://archive.ubuntu.com/ubuntu noble/main amd64 libiscsi7 amd64 1.19.0-3build4 [68.7 kB]  
Get:8 http://archive.ubuntu.com/ubuntu noble/main amd64 libndctl6 amd64 77-2ubuntu2 [62.8 kB]  
Get:9 http://archive.ubuntu.com/ubuntu noble/main amd64 libnfs14 amd64 5.0.2-1build1 [109 kB]
```

سپس دستور رو برای convert کردن به vdi رو میزنیم (به این فرمت تبدیل می کنیم که از آن در box virtual ایمپورت کنیم):

```
p4-vm.qcow2  
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/1$ qemu-img convert -O vdi p4-vm.qcow2 test.vdi
```

مدت زیادی منتظر می مانیم تا تبدیل به فرمت مدنظر شود:

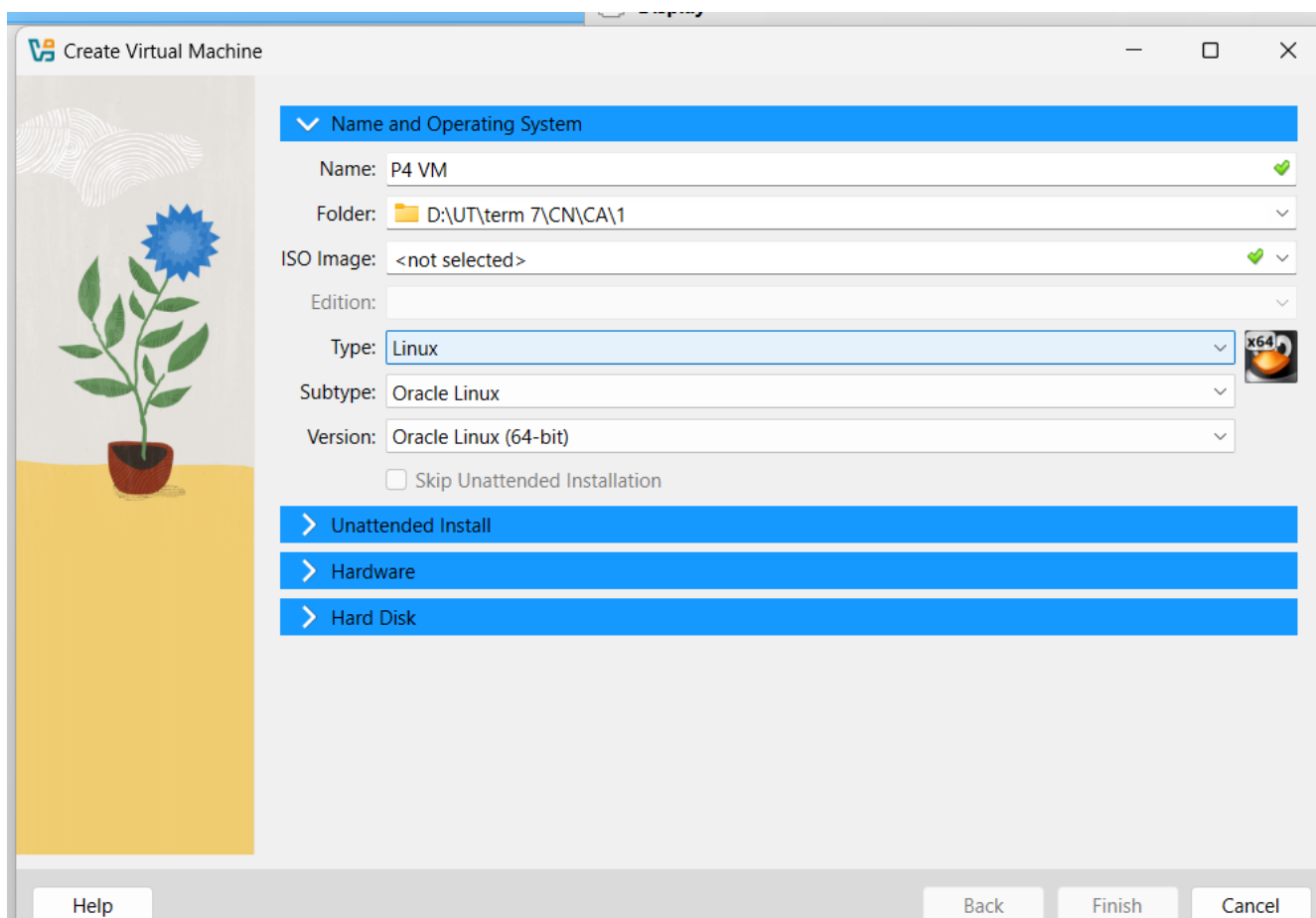
```

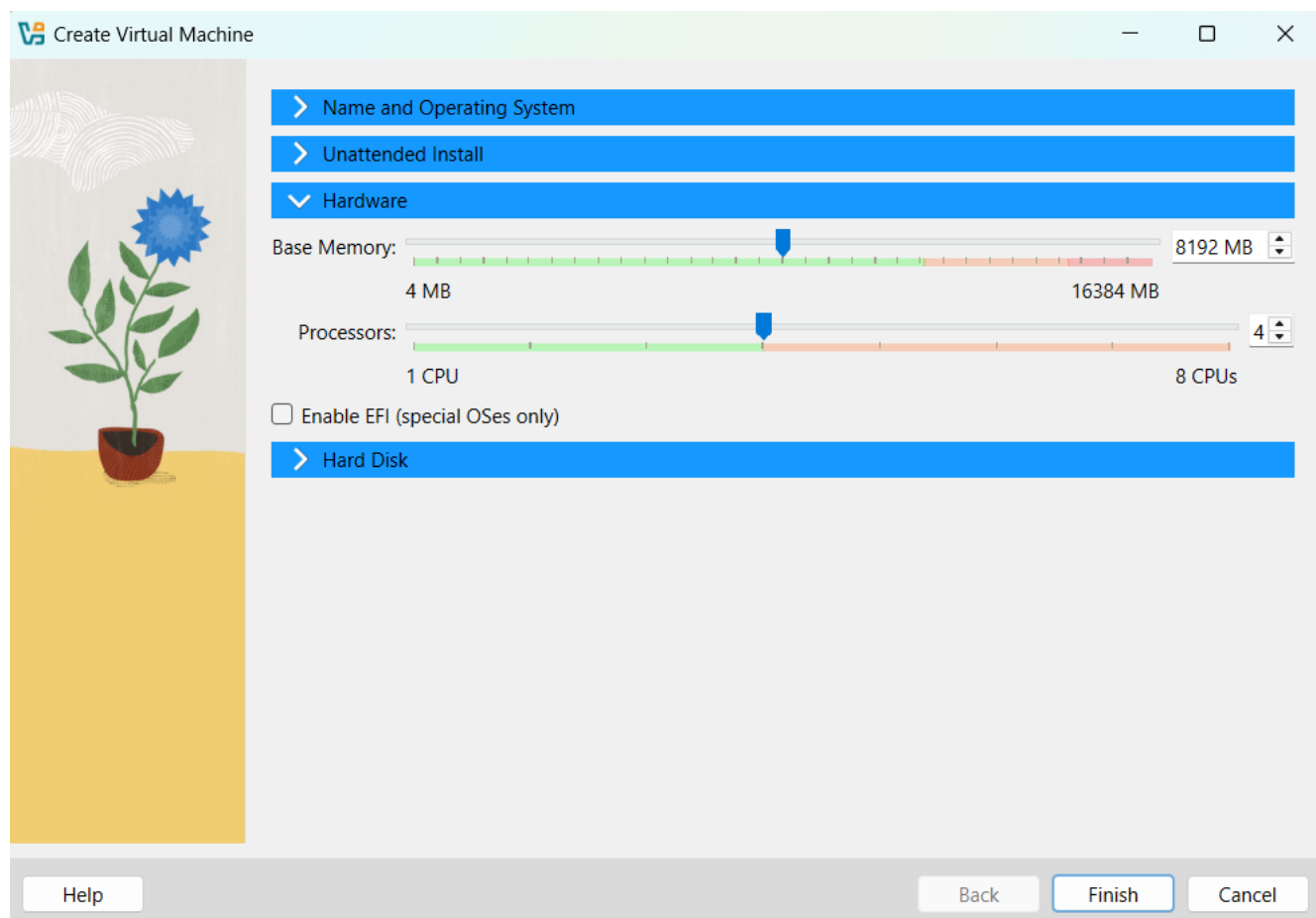
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/1$ mv test.vdi p4-vm.vdi
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/1$ ls
p4-vm.qcow2  p4-vm.vdi
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/1$ ll
total 33021896
drwxr-xr-x 2 mehrad mehrad 4096 Dec 9 19:46 ./
drwxr-xr-x 3 mehrad mehrad 4096 Dec 9 19:14 ../
-rw-r--r-- 1 mehrad mehrad 8598060032 Dec 9 12:34 p4-vm.qcow2
-rw-r--r-- 1 mehrad mehrad 25216340480 Dec 9 19:21 p4-vm.vdi
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/1$ ll -h
total 32G
drwxr-xr-x 2 mehrad mehrad 4.0K Dec 9 19:46 ./
drwxr-xr-x 3 mehrad mehrad 4.0K Dec 9 19:14 ../
-rw-r--r-- 1 mehrad mehrad 8.1G Dec 9 12:34 p4-vm.qcow2
-rw-r--r-- 1 mehrad mehrad 24G Dec 9 19:21 p4-vm.vdi
mehrad@DESKTOP-82AJAD8:~/UT/Term7/CN/CAs/1$

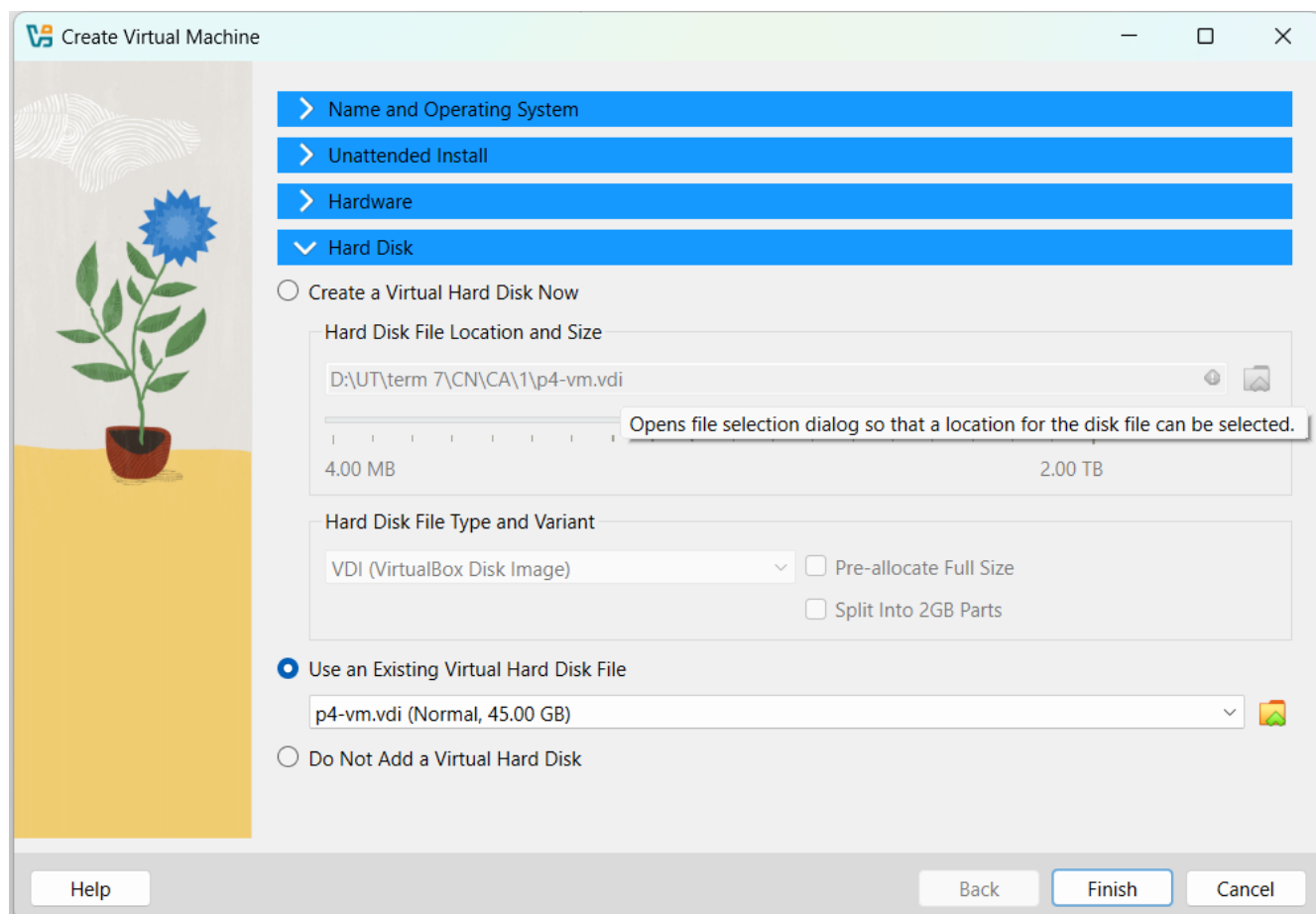
```

حجم آن به ۲۴ گیگ تبدیل شده است.

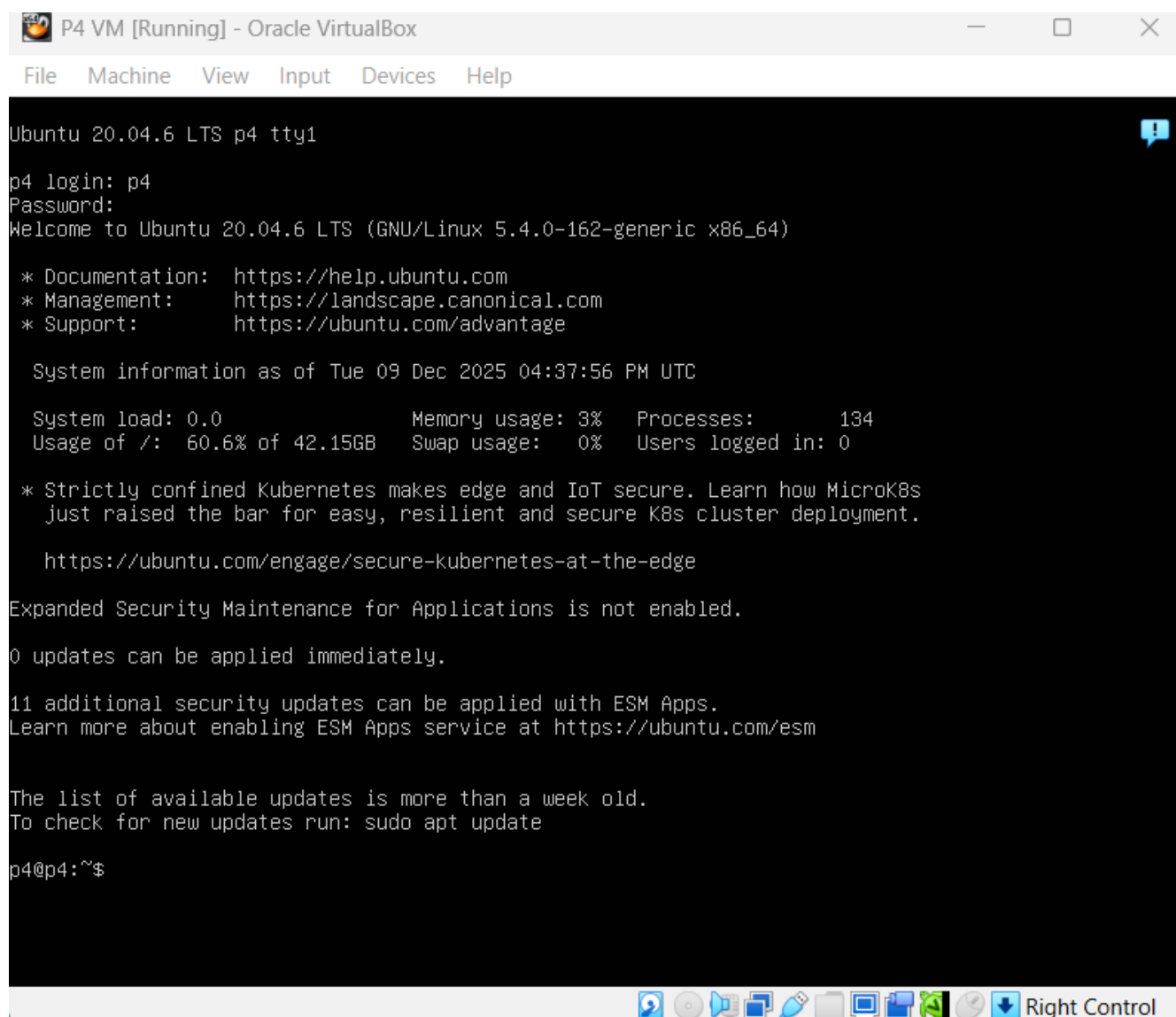
حالا با استفاده از box virtual یک machine virtual جدید میسازیم:







محیط بالا می آید و رمز و نام کاربری رو p4 می زنیم:



```
P4 VM [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Ubuntu 20.04.6 LTS p4 tty1
p4 login: p4
Password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-162-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/advantage

System information as of Tue 09 Dec 2025 04:37:56 PM UTC

System load: 0.0          Memory usage: 3%    Processes:      134
Usage of /:  60.6% of 42.15GB  Swap usage:  0%    Users logged in: 0

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

  https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

11 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

p4@p4:~$
```

مرحله ۱:

به پوشه p4-tools می رویم:

```

P4 VM [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Ubuntu 20.04.6 LTS p4 tty1
p4 login: p4
Password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-162-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Tue 09 Dec 2025 04:37:56 PM UTC

System load: 0.0           Memory usage: 3%   Processes:      134
Usage of /:  60.6% of 42.15GB Swap usage:   0%   Users logged in: 0

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

11 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

p4@p4:~$ ls
mininet  p4-tools
p4@p4:~$ cd
.cache/  .ccache/  .emacs.d/ mininet/  p4-tools/  .ssh/
p4@p4:~$ cd p4-tools/
p4@p4:~/p4-tools$

```

مرحله ۲:

می خواهیم که ریپو رو کلون کنیم ولی به مشکل می خوریم:

```

.cache/  .ccache/  .emacs.d/ mininet/  p4-tools/  .ssh/
p4@p4:~$ cd p4-tools/
p4@p4:~/p4-tools$ git clone https://github.com/nsg-ethz/p4-learning.git
Cloning into 'p4-learning'...
fatal: unable to access 'https://github.com/nsg-ethz/p4-learning.git/': Could not resolve host: github.com
p4@p4:~/p4-tools$ _

```

این مشکل به این خاطر است که دسترسی به اینترنت در machine virtual نداریم.

با استفاده از این دستورات اینترنت را راه می اندازیم (ip میگیریم):

```
p4@p4:~/p4-tools$ sudo ip link set enp0s3 up
p4@p4:~/p4-tools$ sudo dhclient enp0s3
p4@p4:~/p4-tools$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=67.2 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=201 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=54.7 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=60.6 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 54.692/95.849/200.829/60.772 ms
p4@p4:~/p4-tools$ ping -c 4 google.com
PING google.com (216.239.38.120) 56(84) bytes of data.
64 bytes from any-in-2678.1e100.net (216.239.38.120): icmp_seq=1 ttl=255 time=58.9 ms
64 bytes from any-in-2678.1e100.net (216.239.38.120): icmp_seq=2 ttl=255 time=57.6 ms
64 bytes from any-in-2678.1e100.net (216.239.38.120): icmp_seq=3 ttl=255 time=166 ms
64 bytes from any-in-2678.1e100.net (216.239.38.120): icmp_seq=4 ttl=255 time=60.0 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3024ms
rtt min/avg/max/mdev = 57.628/85.737/166.439/46.600 ms
p4@p4:~/p4-tools$
```

حالا دوباره امتحان می کنیم برای کلون کردن:

```
p4@p4:~/p4-tools$ git clone https://github.com/nsg-ethz/p4-learning.git
Cloning into 'p4-learning'...
remote: Enumerating objects: 3136, done.
remote: Counting objects: 100% (82/82), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 3136 (delta 69), reused 64 (delta 64), pack-reused 3054 (from 2)
Receiving objects: 100% (3136/3136), 395.92 MiB | 1.46 MiB/s, done.
Resolving deltas: 100% (1948/1948), done.
Updating files: 100% (751/751), done.
p4@p4:~/p4-tools$ ll
total 52
drwxrwxr-x 13 p4 p4 4096 Dec  9 19:21 ./
drwxr-xr-x  8 p4 p4 4096 Sep 18  2023 ../
drwxrwxr-x 21 p4 p4 4096 Sep 17  2023 bmv2/
drwxrwxr-x 21 p4 p4 4096 Sep 18  2023 bmv2-opt/
drwxrwxr-x 46 p4 p4 4096 Sep 17  2023 frf/
drwxrwxr-x 19 p4 p4 4096 Sep 17  2023 grpc/
drwxrwxr-x 12 p4 p4 4096 Sep 17  2023 libyang/
drwxrwxr-x 18 p4 p4 4096 Sep 17  2023 p4c/
drwxrwxr-x  6 p4 p4 4096 Dec  9 19:26 p4-learning/
drwxrwxr-x 11 p4 p4 4096 Sep 17  2023 p4-utils/
drwxrwxr-x 21 p4 p4 4096 Sep 17  2023 PI/
drwxrwxr-x 24 p4 p4 4096 Sep 17  2023 protobuf/
drwxrwxr-x 11 p4 p4 4096 Sep 17  2023 ptf/
p4@p4:~/p4-tools$ _
```

مرحله ۳:

به فولدر `p4-learning/examples/l2_learning` می رویم:


```
p4@p4:~/p4-tools$ cd p4-learning/examples/l2_learning/  
p4@p4:~/p4-tools/p4-learning/examples/l2_learning$ ll  
total 56  
drwxrwxr-x  3 p4 p4 4096 Dec  9 19:26 ./  
drwxrwxr-x 31 p4 p4 4096 Dec  9 19:26 ../  
-rw-rw-r--  1 p4 p4 4464 Dec  9 19:26 l2_learning_controller.py  
-rw-rw-r--  1 p4 p4 1284 Dec  9 19:26 network_cpu_multi.py  
-rw-rw-r--  1 p4 p4  593 Dec  9 19:26 network_cpu.py  
-rw-rw-r--  1 p4 p4 1253 Dec  9 19:26 network_digest_multi.py  
-rw-rw-r--  1 p4 p4  565 Dec  9 19:26 network_digest.py  
-rw-rw-r--  1 p4 p4  490 Dec  9 19:26 p4app_cpu.json  
-rw-rw-r--  1 p4 p4 1174 Dec  9 19:26 p4app_cpu_multi.json  
-rw-rw-r--  1 p4 p4  455 Dec  9 19:26 p4app_digest.json  
-rw-rw-r--  1 p4 p4 1085 Dec  9 19:26 p4app_digest_multi.json  
drwxrwxr-x  2 p4 p4 4096 Dec  9 19:26 p4src/  
-rw-rw-r--  1 p4 p4 1530 Dec  9 19:26 README.md  
p4@p4:~/p4-tools/p4-learning/examples/l2_learning$
```

مرحله ۴:

دونه به دونه اطلاعات فایل های json رو میبینیم:

اولی با p4app_cpu.json شروع می کنیم:

```

p4app_cpu.json          p4app_digest.json      p4src/
p4app_cpu_multi.json    p4app_digest_multi.json
p4@p4:~/p4-tools/p4-learning/examples/12_learning$ cat p4app_
p4app_cpu.json          p4app_digest.json
p4app_cpu_multi.json    p4app_digest_multi.json
p4@p4:~/p4-tools/p4-learning/examples/12_learning$ cat p4app_cpu.json
{
  "p4_src": "p4src/12_learning_copy_to_cpu.p4",
  "cli": true,
  "pcap_dump": true,
  "enable_log": true,
  "topology": {
    "assignment_strategy": "12",
    "default": {
      "auto_arp_tables": false
    },
    "links": [["h1", "s1"], ["h2", "s1"], ["h3", "s1"], ["h4", "s1"]],
    "hosts": {
      "h1": {
      },
      "h2": {
      },
      "h3": {
      },
      "h4": {
      }
    },
    "switches": {
      "s1": {
        "cpu_port": true
      }
    }
  }
}
p4@p4:~/p4-tools/p4-learning/examples/12_learning$

```

سپس با p4app_cpu_multi.json:

```
"auto_arp_tables": false
},
"links": [
  ["h1", "s1"], ["h2", "s1"], ["h3", "s3"], ["h4", "s3"], ["h5", "s4"],
  ["h6", "s4"], ["s1", "s2"], ["s2", "s3"], ["s2", "s4"]
],
"hosts": {
  "h1": {
  },
  "h2": {
  },
  "h3": {
  },
  "h4": {
  },
  "h5": {
  },
  "h6": {
  }
},
"switches": {
  "s1": {
    "cpu_port": true
  },
  "s2": {
    "cpu_port": true
  },
  "s3": {
    "cpu_port": true
  },
  "s4": {
    "cpu_port": true
  }
}
}
}
b4@p4:~/p4-tools/p4-learning/examples/l2_learning$ _
```



سپس با: p4app_disget.json

```
p4@p4:~/p4-tools/p4-learning/examples/l2_learning$ cat p4app_digest.json
{
  "p4_src": "p4src/l2_learning_digest.p4",
  "cli": true,
  "pcap_dump": true,
  "enable_log": true,
  "topology": {
    "assignment_strategy": "l2",
    "default": {
      "auto_arp_tables": false
    },
    "links": [["h1", "s1"], ["h2", "s1"], ["h3", "s1"], ["h4", "s1"]],
    "hosts": {
      "h1": {
      },
      "h2": {
      },
      "h3": {
      },
      "h4": {
      }
    },
    "switches": {
      "s1": {
      }
    }
  }
}
```

p4@p4:~/p4-tools/p4-learning/examples/l2_learning\$ _



و در آخر هم با p4app_digest_multi.json:

```
],
"topology": {
  "assignment_strategy": "l2",
  "default": {
    "auto_arp_tables": false
  },
  "links": [
    ["h1", "s1"], ["h2", "s1"], ["h3", "s3"], ["h4", "s3"], ["h5", "s4"],
    ["h6", "s4"], ["s1", "s2"], ["s2", "s3"], ["s2", "s4"]
  ],
  "hosts": {
    "h1": {
    },
    "h2": {
    },
    "h3": {
    },
    "h4": {
    },
    "h5": {
    },
    "h6": {
    }
  },
  "switches": {
    "s1": {
    },
    "s2": {
    },
    "s3": {
    },
    "s4": {
    }
  }
}
}
p4@p4:~/p4-tools/p4-learning/examples/l2_learning$
```

مرحله ۵:

اجرای شبکه با p4run با یک cpu:

```
p4@p4:~/p4-tools/p4-learning/examples/l2_learning$ sudo p4run --config p4app_cpu.json
Reading JSON configuration file...
*** Removing excess controllers/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller ovs-
testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller o
vs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([_[:alnum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
brctl show | awk 'FNR > 1 {print $1}'
Compiling P4 files...
```

```
Default interface: h1-eth0      10.0.0.1      00:00:0a:00:00:01
*****
*****
Network configuration for: h2
Default interface: h2-eth0      10.0.0.2      00:00:0a:00:00:02
*****
*****
Network configuration for: h3
Default interface: h3-eth0      10.0.0.3      00:00:0a:00:00:03
*****
*****
Network configuration for: h4
Default interface: h4-eth0      10.0.0.4      00:00:0a:00:00:04
*****
Starting mininet CLI...
```

```
=====
Welcome to the P4 Utils Mininet CLI!
=====
```

```
Your P4 program is installed into the BMV2 software switch
and your initial configuration is loaded. You can interact
with the network using the mininet CLI below.
```

```
To inspect or change the switch configuration, connect to
its CLI from your host operating system using this command:
  simple_switch_CLI --thrift-port <switch thrift port>
```

```
To view a switch log, run this command from your host OS:
  tail -f <log_dir>/<switchname>.log
By default log directory is "./log".
```

```
To view the switch output pcap, check the pcap files in <pcap_dir>:
  for example run:  sudo tcpdump -xxx -r s1-eth1.pcap
By default pcap directory is "./pcap".
```

```
*** Starting CLI:
mininet>
```



خروج:

```
mininet> exit
Stopping network...
*** Stopping 0 controllers

*** Stopping 5 links
.....
*** Stopping 0 routers

*** Stopping 2 switches
s1 sw-cpu

*** Stopping 4 hosts
h1 h2 h3 h4
*** Done
Network stopped!
p4@p4:~/p4-tools/p4-learning/examples/l2_learning$
```



اجرای شبکه با p4run با چند cpu:


```
Default interface: h3-eth0      10.0.0.3      00:00:0a:00:00:03
*****
*****
Network configuration for: h4
Default interface: h4-eth0      10.0.0.4      00:00:0a:00:00:04
*****
*****
Network configuration for: h5
Default interface: h5-eth0      10.0.0.5      00:00:0a:00:00:05
*****
*****
Network configuration for: h6
Default interface: h6-eth0      10.0.0.6      00:00:0a:00:00:06
*****
Starting mininet CLI...
```

```
=====
Welcome to the P4 Utils Mininet CLI!
=====
```

```
Your P4 program is installed into the BMV2 software switch
and your initial configuration is loaded. You can interact
with the network using the mininet CLI below.
```

```
To inspect or change the switch configuration, connect to
its CLI from your host operating system using this command:
  simple_switch_CLI --thrift-port <switch thrift port>
```

```
To view a switch log, run this command from your host OS:
  tail -f <log_dir>/<switchname>.log
By default log directory is "./log".
```

```
To view the switch output pcap, check the pcap files in <pcap_dir>:
  for example run: sudo tcpdump -xxx -r s1-eth1.pcap
By default pcap directory is "./pcap".
```

```
*** Starting CLI:
mininet> _
```



مرحله ۶:

الان شبکه ما با چند cpu بالا اومده و آماده تست کردن است:

```

Default interface: h3-eth0      10.0.0.3      00:00:0a:00:00:03
*****
*****
Network configuration for: h4
Default interface: h4-eth0      10.0.0.4      00:00:0a:00:00:04
*****
*****
Network configuration for: h5
Default interface: h5-eth0      10.0.0.5      00:00:0a:00:00:05
*****
*****
Network configuration for: h6
Default interface: h6-eth0      10.0.0.6      00:00:0a:00:00:06
*****
Starting mininet CLI...

```

```

=====
Welcome to the P4 Utils Mininet CLI!
=====

```

```

Your P4 program is installed into the BMV2 software switch
and your initial configuration is loaded. You can interact
with the network using the mininet CLI below.

```

```

To inspect or change the switch configuration, connect to
its CLI from your host operating system using this command:
  simple_switch_CLI --thrift-port <switch thrift port>

```

```

To view a switch log, run this command from your host OS:
  tail -f <log_dir>/<switchname>.log
By default log directory is "./log".

```

```

To view the switch output pcap, check the pcap files in <pcap_dir>:
  for example run: sudo tcpdump -xxx -r s1-eth1.pcap
By default pcap directory is "./pcap".

```

```

*** Starting CLI:
mininet> _

```



مرحله ۷:

حالا به ترتیب تک تک دستور های گفته شده رو انجام میدهم:

وقتی فقط یک cpu رو امتحان می کنیم:

دستور all: ping

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X
h2 -> X X X
h3 -> X X X
h4 -> X X X
*** Results: 100% dropped (0/12 received)
mininet> _
```

همانطور که مشخص است همه بسته های از سال شده از هر هاست به ۳ تای دیگر loss شده است.
دستور ping بین دو هاست:

```
*** Results: 100% dropped (0/12 received)
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable
From 10.0.0.1 icmp_seq=4 Destination Host Unreachable
From 10.0.0.1 icmp_seq=5 Destination Host Unreachable
From 10.0.0.1 icmp_seq=6 Destination Host Unreachable
From 10.0.0.1 icmp_seq=7 Destination Host Unreachable
From 10.0.0.1 icmp_seq=8 Destination Host Unreachable
From 10.0.0.1 icmp_seq=9 Destination Host Unreachable
^C
--- 10.0.0.2 ping statistics ---
10 packets transmitted, 0 received, +9 errors, 100% packet loss, time 9219ms
pipe 4
mininet> _
```

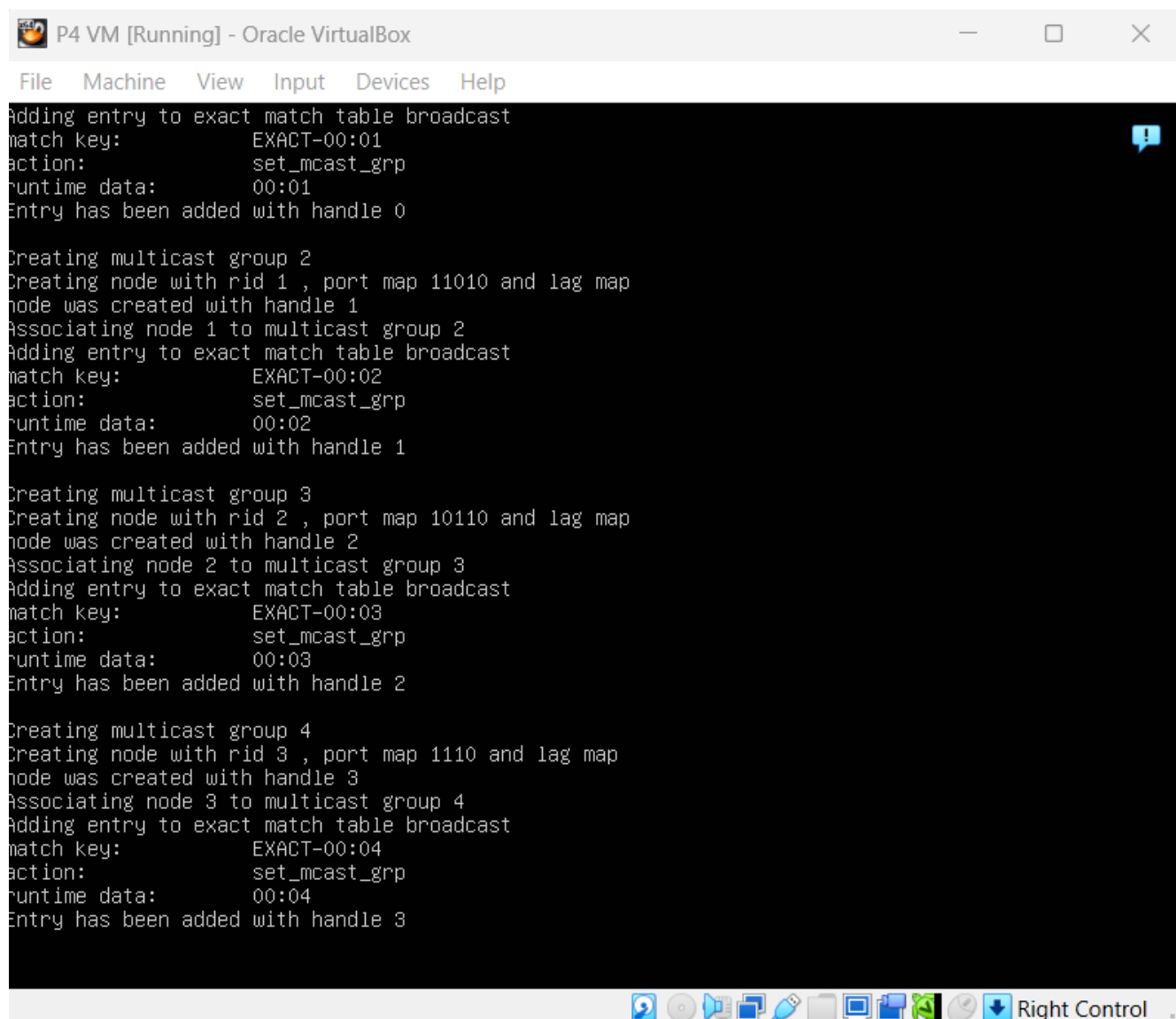
همانطور که مشخص است همه درخواست های ارسال شده همشون loss شدن و نرسیدن.
دستور متصل شدن به یک هاست:

```
mininet> h1 python3
Python 3.8.10 (default, May 26 2023, 14:05:08)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 1+2
3
>>> 2*4
8
>>> exit
Use exit() or Ctrl-D (i.e. EOF) to exit
>>> ^[[A^?^?^?
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'e' is not defined
>>> exit()
mininet>
```

چند دستور رو امتحان کردیم و درست بود و موفقیت آمیز به آن هاست متصل شدیم. در ابتدای اجرای آزمایش، بعد از راه اندازی توپولوژی شبکه و اجرای دستور pingall، مشخص شد که هیچ کدوم از هاست ها نمی تونن با هم ارتباط برقرار کنن و همه بسته های ارسالی با loss صد درصد روبه رو می شن. بررسی ها نشون داد که با این که بسته ها به سوئیچ ها می رسن، اما چون جداول سوئیچ خالی هستن و کنترلر به صورت خودکار اجرا نشده، سوئیچ نمی تونه تصمیم بگیره هر بسته رو به کدوم پورت بفرسته و در نتیجه همه بسته ها drop می شن. برای رفع این مشکل، اسکریپت کنترلر learning به صورت دستی و در یک ترمینال جدا اجرا می کنیم. این اسکریپت که داخل فایل l2_learning_controller.py قرار داره، وظیفه داره پیام های Digest شامل آدرس MAC و پورت ورودی رو از سوئیچ دریافت کنه و بر اساس اون ها، قوانین لازم برای ارسال بسته ها رو داخل جداول سوئیچ نصب کنه. برای اجرای کنترلر، دستور زیر اجرا شد:

```
cpu s1 l2_learning_controller.py python sudo
```

اینم نتیجه ش:



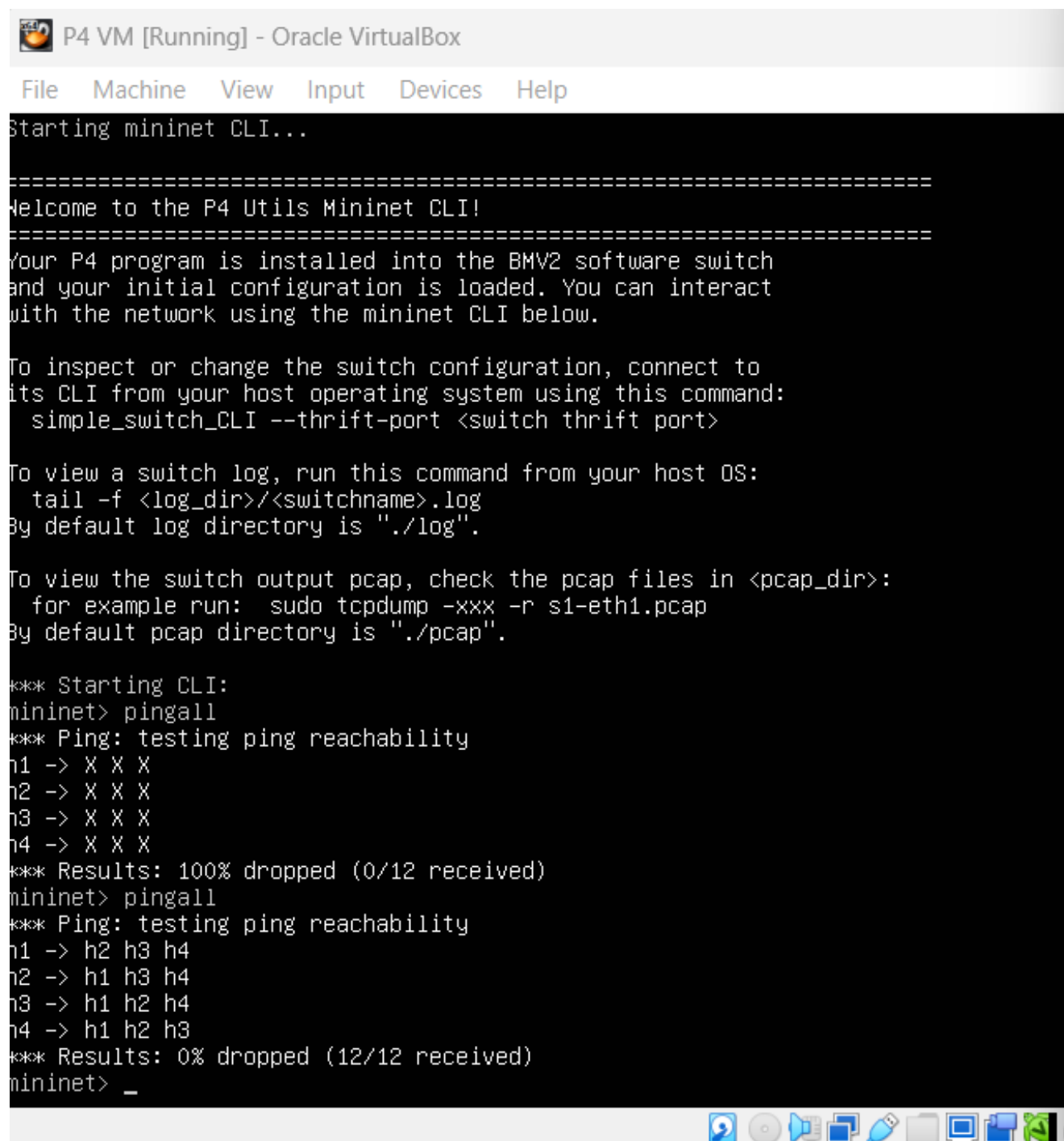
```
File Machine View Input Devices Help
Adding entry to exact match table broadcast
match key:      EXACT-00:01
action:         set_mcast_grp
runtime data:   00:01
Entry has been added with handle 0

Creating multicast group 2
Creating node with rid 1 , port map 11010 and lag map
node was created with handle 1
Associating node 1 to multicast group 2
Adding entry to exact match table broadcast
match key:      EXACT-00:02
action:         set_mcast_grp
runtime data:   00:02
Entry has been added with handle 1

Creating multicast group 3
Creating node with rid 2 , port map 10110 and lag map
node was created with handle 2
Associating node 2 to multicast group 3
Adding entry to exact match table broadcast
match key:      EXACT-00:03
action:         set_mcast_grp
runtime data:   00:03
Entry has been added with handle 2

Creating multicast group 4
Creating node with rid 3 , port map 1110 and lag map
node was created with handle 3
Associating node 3 to multicast group 4
Adding entry to exact match table broadcast
match key:      EXACT-00:04
action:         set_mcast_grp
runtime data:   00:04
Entry has been added with handle 3
```

بعد از فعال شدن کنترلر و نصب قوانین forwarding، ارتباط بین هاست‌ها به درستی برقرار شد و اجرای دوباره دستور pingall نشون داد که دیگه هیچ بسته‌ای از دست نمی‌ره.
این هم عکس هاش:



The screenshot shows a terminal window titled "P4 VM [Running] - Oracle VirtualBox". The terminal displays the following text:

```
Starting mininet CLI...

=====
Welcome to the P4 Utils Mininet CLI!
=====

Your P4 program is installed into the BMV2 software switch
and your initial configuration is loaded. You can interact
with the network using the mininet CLI below.

To inspect or change the switch configuration, connect to
its CLI from your host operating system using this command:
  simple_switch_CLI --thrift-port <switch thrift port>

To view a switch log, run this command from your host OS:
  tail -f <log_dir>/<switchname>.log
By default log directory is "./log".

To view the switch output pcap, check the pcap files in <pcap_dir>:
  for example run: sudo tcpdump -xxx -r s1-eth1.pcap
By default pcap directory is "./pcap".

*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X
h2 -> X X X
h3 -> X X X
h4 -> X X X
*** Results: 100% dropped (0/12 received)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet> _
```

The terminal window has a menu bar with "File", "Machine", "View", "Input", "Devices", and "Help". The bottom of the window shows a taskbar with various icons.

```

P4 VM [Running] - Oracle VirtualBox
File Machine View Input Devices Help

simple_switch_CLI --thrift-port <switch thrift port>

To view a switch log, run this command from your host OS:
tail -f <log_dir>/<switchname>.log
By default log directory is "./log".

To view the switch output pcap, check the pcap files in <pcap_dir>:
for example run: sudo tcpdump -xxx -r s1-eth1.pcap
By default pcap directory is "./pcap".

ok Starting CLI:
mininet> pingall
ok Ping: testing ping reachability
h1 -> X X X
h2 -> X X X
h3 -> X X X
h4 -> X X X
ok Results: 100% dropped (0/12 received)
mininet> pingall
ok Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
ok Results: 0% dropped (12/12 received)
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
4 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.79 ms
4 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=2.98 ms
4 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=3.82 ms
4 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=1.38 ms
4 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=3.78 ms
C
-- 10.0.0.2 ping statistics --
5 packets transmitted, 5 received, 0% packet loss, time 4013ms
rtt min/avg/max/mdev = 1.375/2.748/3.815/1.006 ms
mininet> _

```

وقتی فقط چند cpu رو امتحان می کنیم:

دستور: pingall

```
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6
h2 -> h1 h3 h4 h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
*** Results: 0% dropped (30/30 received)
mininet>
```



برای هر کدوم از هاست ها به ۵ هاست دیگر یک درخواست ارسال کرده که همشون رسیده و هیچ کدوم دراپ نشده است.

دستور ping بین دو هاست:


```
*** Results: 0% dropped (30/30 received)
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.45 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=4.85 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=5.67 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=5.41 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=5.21 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=5.63 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=5.52 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=5.56 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=4.68 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=5.12 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=6.21 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=6.11 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=6.28 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=5.00 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=5.02 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=7.80 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=5.81 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=5.01 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=4.92 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=2.04 ms
64 bytes from 10.0.0.2: icmp_seq=21 ttl=64 time=2.87 ms
^C
--- 10.0.0.2 ping statistics ---
21 packets transmitted, 21 received, 0% packet loss, time 20072ms
rtt min/avg/max/mdev = 2.036/5.101/7.802/1.276 ms
mininet>
```

همانطور که مشخص است از هاست ۱ به هاست ۲ به تعداد ۲۱ بار درخواست داده شده است و همشون سالم رسیده است و loss نداشتیم میانگین تاخیر هم ۵ میلی ثانیه هست تقریباً.
دستور متصل شدن به یک هاست:

```
mininet> h1 python3
Python 3.8.10 (default, May 26 2023, 14:05:08)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 1+2
3
>>> 2*4
8
>>> ^D
mininet> _
```

همانطور که در تصویر مشخص است به هاست اول متصل شدیم و در آن جا در محیط پایتون دو دستور انجام دادیم برای تست.

مرحله ۸ (امتیازی):

با استفاده از کنترل راست و F2 یک ترمینال جدید باز می کنیم و رمز و نام کاربری رو p4 میزنیم:

```
p4 login: p4
Password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-162-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Tue 09 Dec 2025 08:20:03 PM UTC

System load:          0.2
Usage of /:           63.4% of 42.15GB
Memory usage:         8%
Swap usage:           0%
Processes:            180
Users logged in:      1
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe5f:d7f5

Expanded Security Maintenance for Infrastructure is not enabled.

251 updates can be applied immediately.
197 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

94 additional security updates can be applied with ESM Infra.
Learn more about enabling ESM Infra service for Ubuntu 20.04 at
https://ubuntu.com/20-04

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection
or proxy settings

Last login: Tue Dec  9 20:04:47 UTC 2025 on tty2
p4@p4:~$
```



فولدر tools در path ای که مشخص شده بود وجود نداشت. برای فهمیدن اینکه این فایل کجاست از دستور زیر استفاده کردیم:

```
p4@p4:~/p4-tools/p4-learning$ cd ..  
p4@p4:~/p4-tools$ cd p4-  
p4-learning/ p4-utils/  
p4@p4:~/p4-tools$ find ~/p4-tools -name runtime_CLI.py  
/home/p4/p4-tools/bmv2-opt/tools/runtime_CLI.py  
/home/p4/p4-tools/bmv2/tools/runtime_CLI.py  
p4@p4:~/p4-tools$ _
```



به آدرس این فایل پایتون میرویم (دومی):

```
p4@p4:~/p4-tools$ cd bmv2/t  
targets/      tests/      third_party/ thrift_src/  tools/  
p4@p4:~/p4-tools$ cd bmv2/tools/  
p4@p4:~/p4-tools/bmv2/tools$
```



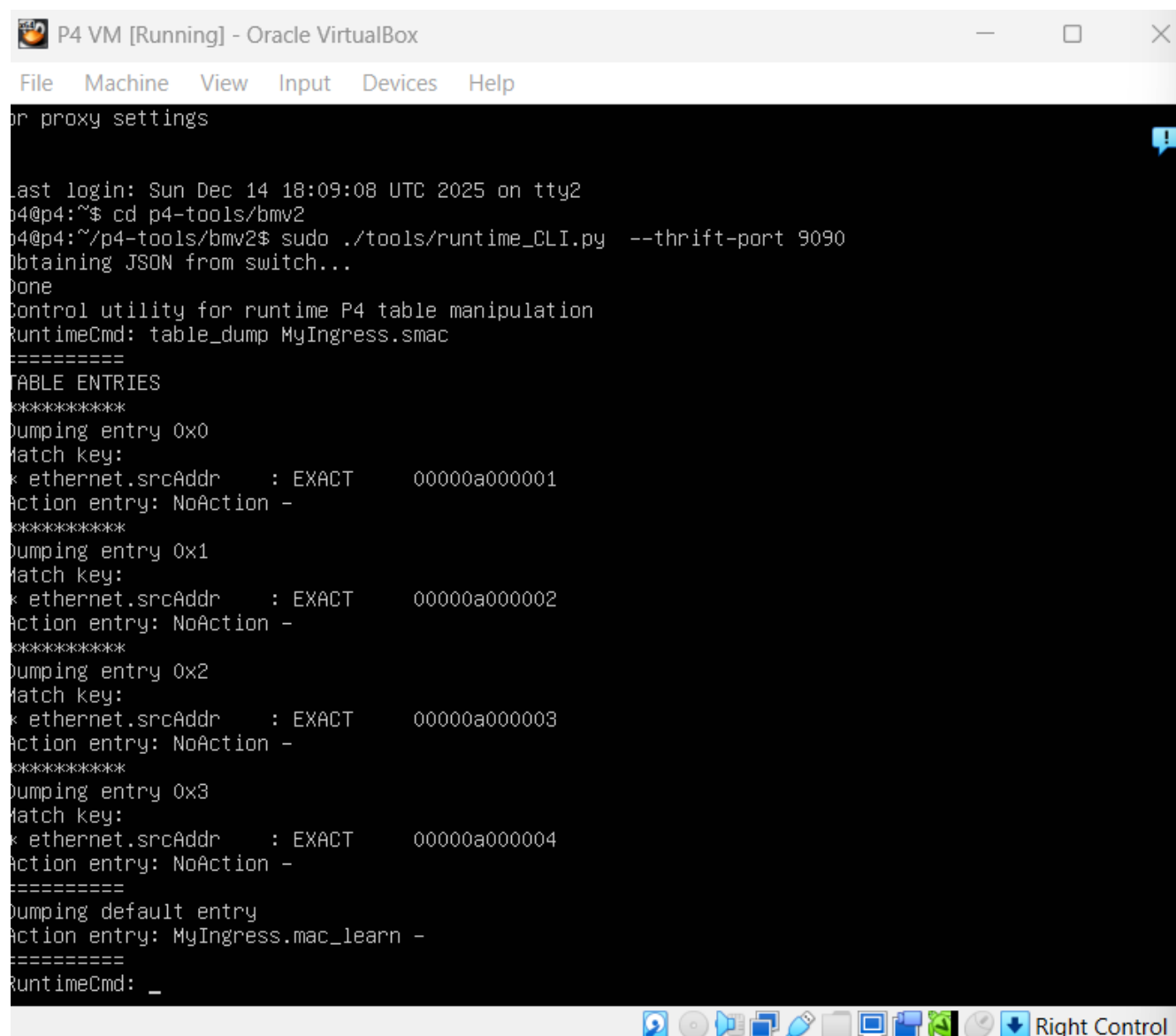
دستور گفته شده رو با پورت مشخص شده امتحان می کنیم:

```
RuntimeCmd:
p4@p4:~/p4-tools/bmv2/tools$ sudo ./runtime_CLI.py --thrift-port 9090
Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
RuntimeCmd: _
```

با استفاده از دستور show_tables جدول ها رو میبینیم:

```
Control utility for runtime P4 table manipulation
RuntimeCmd: show_tables
MyIngress.broadcast [implementation=None, mk=standard_metadata.ingress_port(exact, 9)]
MyIngress.dmac [implementation=None, mk=ethernet.dstAddr(exact, 48)]
MyIngress.smac [implementation=None, mk=ethernet.srcAddr(exact, 48)]
tbl_l2_learning_copy_to_cpu153 [implementation=None, mk=]
RuntimeCmd: _
```

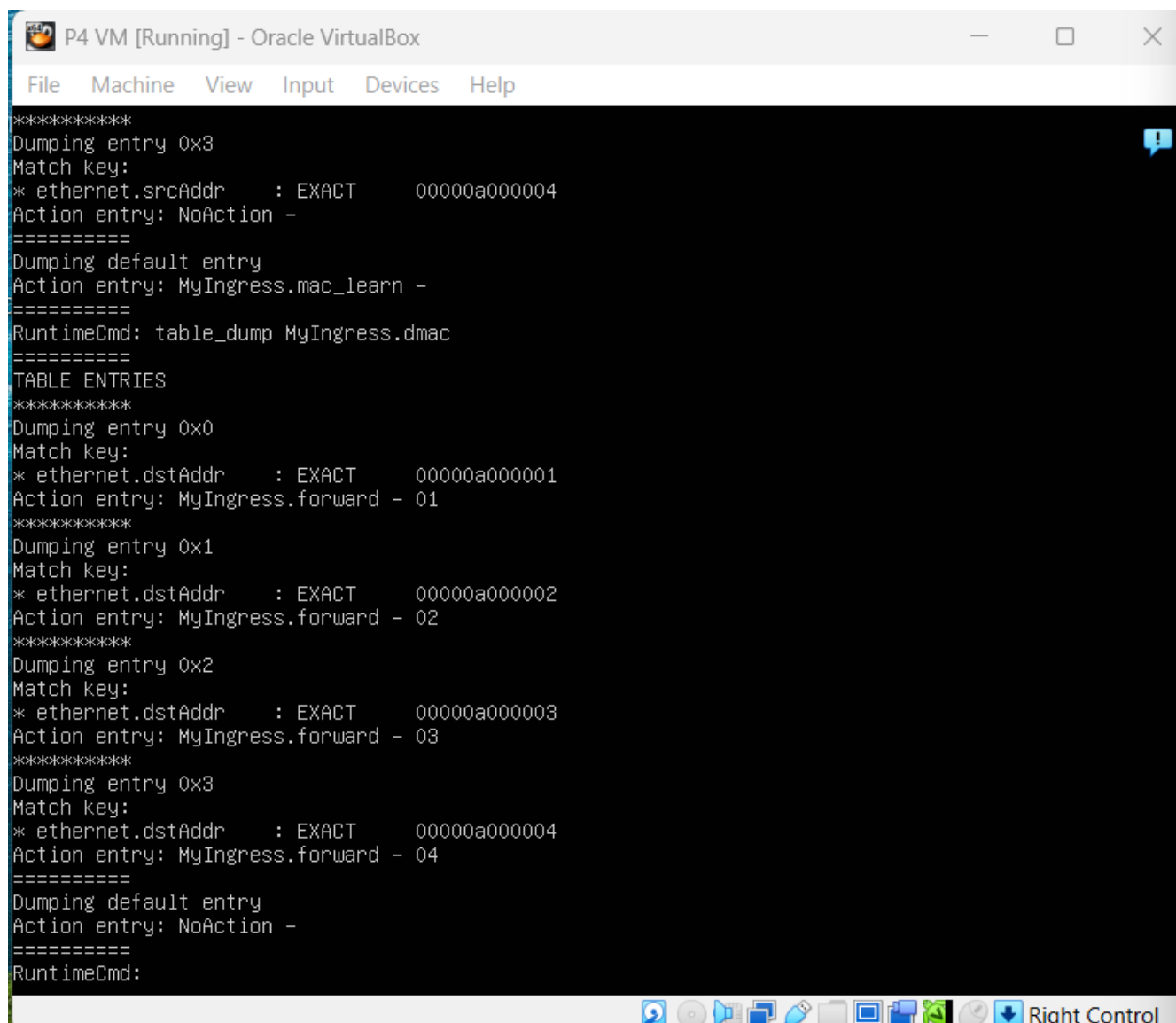
حالا می توانیم با دستور table_dump اطلاعات یک جدول رو ببینیم:



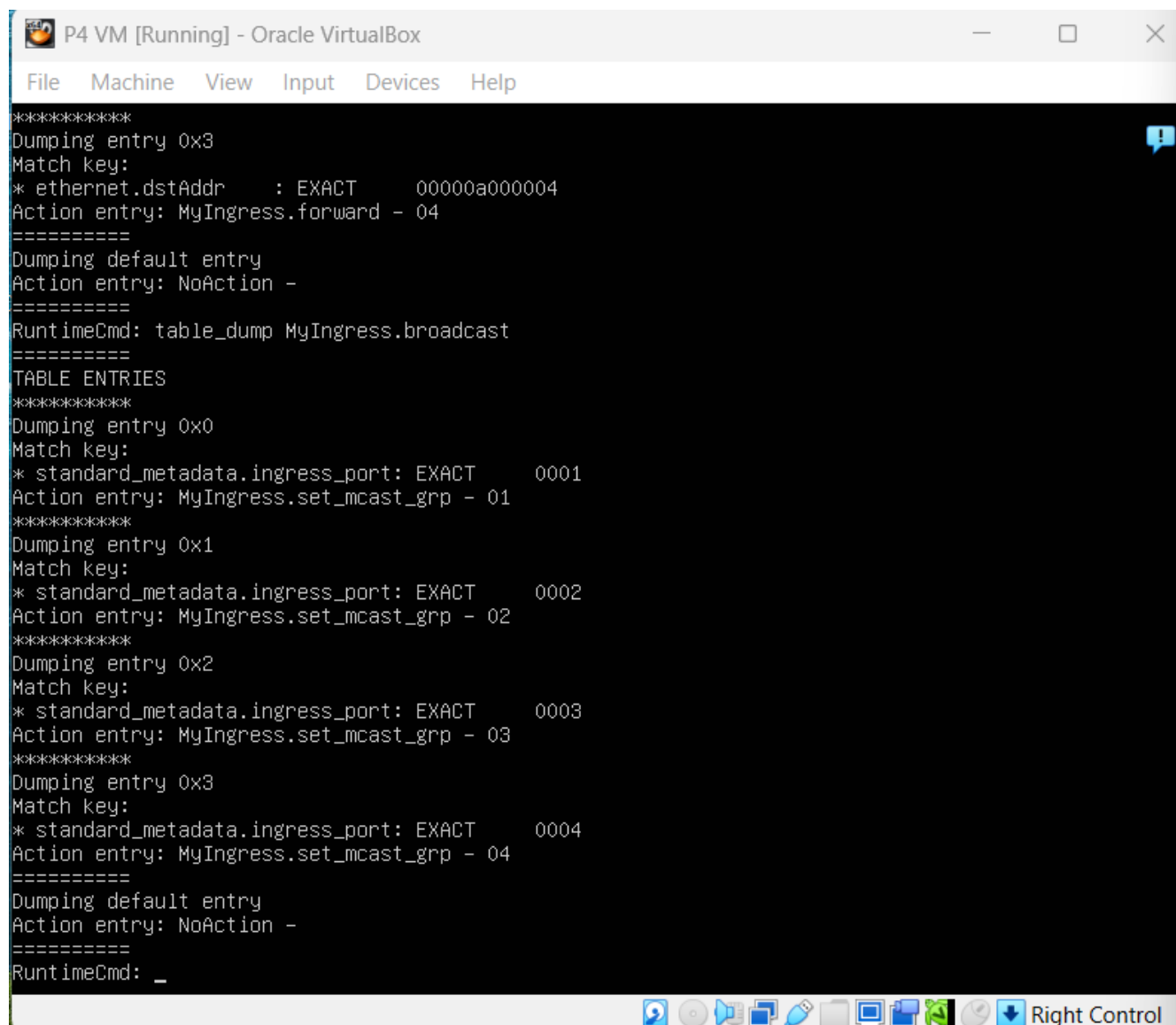
```
P4 VM [Running] - Oracle VirtualBox
File Machine View Input Devices Help

or proxy settings

Last login: Sun Dec 14 18:09:08 UTC 2025 on tty2
p4@p4:~$ cd p4-tools/bmv2
p4@p4:~/p4-tools/bmv2$ sudo ./tools/runtime_CLI.py --thrift-port 9090
Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
RuntimeCmd: table_dump MyIngress.smac
=====
TABLE ENTRIES
*****
Dumping entry 0x0
Match key:
* ethernet.srcAddr      : EXACT      00000a000001
Action entry: NoAction -
*****
Dumping entry 0x1
Match key:
* ethernet.srcAddr      : EXACT      00000a000002
Action entry: NoAction -
*****
Dumping entry 0x2
Match key:
* ethernet.srcAddr      : EXACT      00000a000003
Action entry: NoAction -
*****
Dumping entry 0x3
Match key:
* ethernet.srcAddr      : EXACT      00000a000004
Action entry: NoAction -
=====
Dumping default entry
Action entry: MyIngress.mac_learn -
=====
RuntimeCmd: _
```



```
*****
Dumping entry 0x3
Match key:
* ethernet.srcAddr : EXACT 00000a000004
Action entry: NoAction -
=====
Dumping default entry
Action entry: MyIngress.mac_learn -
=====
RuntimeCmd: table_dump MyIngress.dmac
=====
TABLE ENTRIES
*****
Dumping entry 0x0
Match key:
* ethernet.dstAddr : EXACT 00000a000001
Action entry: MyIngress.forward - 01
*****
Dumping entry 0x1
Match key:
* ethernet.dstAddr : EXACT 00000a000002
Action entry: MyIngress.forward - 02
*****
Dumping entry 0x2
Match key:
* ethernet.dstAddr : EXACT 00000a000003
Action entry: MyIngress.forward - 03
*****
Dumping entry 0x3
Match key:
* ethernet.dstAddr : EXACT 00000a000004
Action entry: MyIngress.forward - 04
=====
Dumping default entry
Action entry: NoAction -
=====
RuntimeCmd:
```



The screenshot shows a terminal window titled "P4 VM [Running] - Oracle VirtualBox". The terminal output displays a series of commands and their results, including dumping entries for a table, matching keys, and setting ingress ports. The output is as follows:

```
*****
Dumping entry 0x3
Match key:
* ethernet.dstAddr : EXACT 00000a000004
Action entry: MyIngress.forward - 04
=====
Dumping default entry
Action entry: NoAction -
=====
RuntimeCmd: table_dump MyIngress.broadcast
=====
TABLE ENTRIES
*****
Dumping entry 0x0
Match key:
* standard_metadata.ingress_port: EXACT 0001
Action entry: MyIngress.set_mcast_grp - 01
*****
Dumping entry 0x1
Match key:
* standard_metadata.ingress_port: EXACT 0002
Action entry: MyIngress.set_mcast_grp - 02
*****
Dumping entry 0x2
Match key:
* standard_metadata.ingress_port: EXACT 0003
Action entry: MyIngress.set_mcast_grp - 03
*****
Dumping entry 0x3
Match key:
* standard_metadata.ingress_port: EXACT 0004
Action entry: MyIngress.set_mcast_grp - 04
=====
Dumping default entry
Action entry: NoAction -
=====
RuntimeCmd: _
```

The window includes a menu bar with "File", "Machine", "View", "Input", "Devices", and "Help". The bottom status bar shows various icons and the text "Right Control".


```

Action entry: NoAction -
=====
RuntimeCmd: table_dump MyIngress.broadcast
=====
TABLE ENTRIES
*****
Dumping entry 0x0
Match key:
* standard_metadata.ingress_port: EXACT      0001
Action entry: MyIngress.set_mcast_grp - 01
*****
Dumping entry 0x1
Match key:
* standard_metadata.ingress_port: EXACT      0002
Action entry: MyIngress.set_mcast_grp - 02
*****
Dumping entry 0x2
Match key:
* standard_metadata.ingress_port: EXACT      0003
Action entry: MyIngress.set_mcast_grp - 03
*****
Dumping entry 0x3
Match key:
* standard_metadata.ingress_port: EXACT      0004
Action entry: MyIngress.set_mcast_grp - 04
=====
Dumping default entry
Action entry: NoAction -
=====
RuntimeCmd: table_dump tbl_l2_learning_copy_to_cpu153
=====
TABLE ENTRIES
=====
Dumping default entry
Action entry: 12_learning_copy_to_cpu153 -
=====
RuntimeCmd:

```

جدول MyIngress.smac برای یاد گرفتن آدرس MAC مبدأ بسته ها استفاده می شه و وقتی محتویاتش رو بررسی می کنیم، می بینیم که آدرس MAC همه هاست هایی که تا اینجا بسته فرستادن داخلش ثبت شده و برای هر کدام اکشن NoAction قرار گرفته، یعنی سوئیچ این آدرس ها رو قبلاً یاد گرفته و دیگه لازم نیست بابت دیدن دوباره شون پیام Digest به کنترلر بفرسته. جدول MyIngress.dmac مربوط به آدرس MAC مقصد هست و اطلاعات داخلش نشون می ده که کنترلر تونسته برای هر MAC مقصد، قانون forward مناسب رو با پورت خروجی درست داخل سوئیچ نصب کنه تا بسته ها به مقصد صحیح هدایت بشن. در کنار این دو جدول، جدول MyIngress.broadcast و همچنین جدول tbl_l2_copy_to_cpu153 نقش پشتیبان رو دارن؛ به این صورت که MyIngress.broadcast مشخص می کنه در صورت ناشناخته بودن مقصد، بسته ها چطور به صورت flooding و به همه پورت ها به جز پورت ورودی ارسال بشن،

و جدول cpu-to-copy-learning-l2-tbl هم تضمین می‌کند هر وقت سوئیچ با یک MAC جدید روبه‌رو می‌شود، یک نمونه از بسته برای یادگیری به CPU و کنترلر فرستاده بشود تا فرآیند یادگیری MAC به‌درستی انجام بشود.

توضیحات و رپورت تکمیلی

۱) راه‌اندازی VM و تبدیل فایل برای VirtualBox

فایل ایمیج اولیه معمولاً با پسوند qcow2 می‌آید. چون VirtualBox راحت‌تر با vdi کار می‌کند، باید یک بار تبدیلش کنیم. روی Ubuntu (یا داخل خود سیستم‌عامل میزبان)، با نصب qemu-utils این تبدیل انجام می‌شود:

```
sudo apt-get install qemu-utils
qemu-img convert -O vdi <input>.qcow2 <output>.vdi
```

روی Windows هم همین کار با qemu-img.exe انجام می‌شود (مسیرها بسته به جایی که فایل‌ها رو گذاشتید عوض می‌شن):

```
qemu-img.exe convert -O vdi D:\...\p4-utils-vm.qcow2 D:\...\p4-utils-vm.vdi
```

بعدش داخل VirtualBox یک Virtual Machine جدید می‌سازیم، نوع سیستم‌عامل رو Ubuntu (64-bit) می‌ذاریم و گزینه‌ی Use an existing virtual hard disk رو انتخاب می‌کنیم و فایل vdi رو بهش می‌دیم.

نکته‌ی آپدیت و مخزن‌ها. گاهی موقع apt-get update / apt update به ارور مخزن یا update می‌خوریم. یک راه رایج (همون چیزی که توی صورت پروژه هم اشاره شده) اینه که از Settings برید Software & Updates و توی بخش Other Software بعضی repositoryها رو موقتاً disable کنید، بعد دوباره update رو بزنید.

نکته‌ی خطای vboxdrv / VirtualBox. اگر موقع start کردن VM با ارور مربوط به vboxdrv یا kernel module روبه‌رو شدید، معمولاً این چند دستور کمک می‌کند (بعدش یک reboot لازم می‌شه):

```
sudo modprobe vboxdrv
sudo apt update
sudo apt install --reinstall linux-headers-$(uname -r) virtualbox-dkms dkms
```

و اگر Secure Boot روشن باشه، بعضی وقت‌ها باید از BIOS/UEFI خاموشش کنید تا ماژول‌ها لود بشن.

۲) دانلود پروژه و اجرای مثال l2_learning

طبق صورت پروژه، مسیر کلی کار اینه که اول وارد فولدر ابزارها می‌شیم و ریپوی مثال‌ها رو از GitHub می‌گیریم:

```
cd ~/p4-tools
```

```
git clone https://github.com/nsg-ethz/p4-learning.git
```

بعد می‌ریم داخل مثال l2_learning:

```
cd ~/p4-tools/p4-learning/examples/l2_learning
```

نکته‌ی مهم اینه که دستور p4run رو بهتره دقیقاً از همین دایرکتوری اجرا کنیم تا فایل‌های JSON درست پیدا بشن.

این فایل‌های JSON دقیقاً چی می‌گن؟ این فایل‌ها هم توپولوژی Mininet رو مشخص می‌کنن (تعداد host و switch) و هم این‌که هر switch با چه P4 program بالا بیاد، پورت‌های مدیریتی / Thrift چی باشه، و آیا اسکریپت‌های controller خودکار اجرا بشن یا نه. برای همین هم تفاوت سناریوهای cpu و multi در عمل از همین جا میاد.

۳) مشکل Network is unreachable داخل VM و راه‌حل

یک مشکلی که بود اینه که گاهی داخل VM اینترنت نداریم و موقع clone کردن خطای Network is unreachable می‌خوریم. معمولاً علت اینه که کارت شبکه (مثلاً enp0s3) بالا نیومده یا Network Manager درست DHCP نگرفته. راه‌حل دستی‌اش اینه:

```
sudo ip link set enp0s3 up
```

```
sudo dhclient enp0s3
```

خط اول رابط رو در لایه‌ی لینک فعال می‌کنه و خط دوم از DHCP یک IP و Gateway می‌گیره؛ بعدش معمولاً git clone و نصب پکیج‌ها بدون مشکل انجام می‌شن.

۴) سناریوی تک‌سوئیچ: چرا اولش pingall صددرصد drop می‌شه؟

وقتی سناریوی ساده رو با چیزی شبیه به دستور زیر بالا میاریم:

```
sudo p4run -config p4app_cpu.json
```

در شروع کار، Data Plane هنوز «چیزی یاد نگرفته» و جدول های L2 forwarding خالی هستند. بنابراین وقتی بسته های اولیه می رسن، سوئیچ نمی دونه برای مقصدهای ناشناس باید به کدوم پورت forward کنه و نتیجه اش این می شه که pingall عملاً loss صددرصد می ده.

اینجا نقش Control Plane خیلی مهمه: کنترلر باید از طریق پیام های Digest (یا مکانیزم مشابه) آدرس های MAC و پورت ورودی رو یاد بگیره و به محض یاد گرفتن، rule های لازم رو داخل جدول ها نصب کنه. اگر کنترلر خودکار اجرا نشه (یا به هر دلیلی درست بالا نیاد)، مجبوریم دستی اجراش کنیم (همون کاری که بالاتر هم انجام دادیم)، مثلاً:

```
sudo python l2_learning_controller.py s1 cpu
```

بعد از این مرحله، با چندتا ping ساده، کنترلر آدرس ها رو یاد می گیره، جدول ها populate می شن و دوباره pingall بدون drop اجرا می شه.

۵) سناریوی چندسوئیچ/چند CPU: تفاوتش با حالت ساده

برای سناریوی چندسوئیچ معمولاً چیزی شبیه این اجرا می کنیم:

```
sudo p4run -config p4app_cpu_multi.json
```

تفاوت اصلی اینه که در این حالت، در خود فایل JSON تعریف شده که برای هر سوئیچ یک controller process جدا بالا بیاد (یعنی لازم نیست دستی ترمینال های مختلف باز کنیم). توپولوژی هم معمولاً شبیه حالت درختی/ aggregation می شه (چند سوئیچ + چند هاست). نتیجه ی عملی اش اینه که شبکه سریع تر «یاد می گیره» و در تست هایی مثل pingall، drop نزدیک صفر می مونه.

۶) بخش امتیازی: چک کردن جدول ها با runtime_CLI.py و معنی خروجی ها

برای این که مطمئن بشیم کنترلر واقعاً rule نصب کرده، از runtime_CLI به پورت مدیریتی سوئیچ وصل می شیم (پورت رایج 9090 هست، ولی دقیقش رو می شه از JSON هم دید):

```
sudo ./tools/runtime_CLI.py -thrift-port 9090
```

(اگر مسیر tools متفاوت بود، مثل کاری که انجام دادیم، با find می شه فایل runtime_CLI.py رو پیدا کرد). بعدش با show_tables لیست جدول ها رو می بینیم و با table_dump <table> (یا در بعضی نسخه ها با دستوری مثل <table> dump-table) محتوای هر جدول رو می ریزیم بیرون.

چه چیزهایی انتظار داریم ببینیم؟

□ جدول MyIngress.smac: آدرس های MAC مبدایی که دیده شده اند اینجا ثبت می شن. اکشن NoAction عمداً گذاشته می شه تا سوئیچ بفهمه این MAC قبلاً دیده شده و لازم نیست برای هر بسته ی تکراری دوباره Digest به کنترلر بفرسته (یعنی بار Control Plane کمتر می شه).

□ جدول MyIngress.dmac: این جدول نگاشت egress port -> MAC destination رو نگه می داره. این همون چیزیه که باعث می شه بعد از یادگیری، بسته ها unicast و درست forward بشن. معمولاً match type از نوع EXACT هست.

□ جدول MyIngress.broadcast: برای وقتی که مقصد ناشناخته است، منطق flooding رو کنترل می کنه و معمولاً با multicast group کاری می کنه که بسته به همه ی پورت ها به جز پورت ورودی بره (همون ایده ی split horizon).

□ جدول/اکشن cpu-to-copy-learning-l2-tbl (یا نام های مشابه): اگر ورودی خاصی هم نداشته باشه، اکشن پیش فرضش باعث می شه وقتی یک MAC جدید دیده شد، یک copy از اطلاعات لازم به سمت CPU/کنترلر ارسال بشه تا چرخه ی یادگیری کامل بشه.

مراجع